bulk_extractor

streaming media analysis

The First Question

You receive a 500 GB drive

Q: What data is on this media?

- Traditional approach: mount, browse, index / keyword search
  - slow, manual, depends on intact filesystem metadata
  - misses deleted files, slack space, swap, compressed data
- Needed: a tool that scans everything, fast, with zero assumptions

➔ bulk_extractor performs **stream-based scanning** — no filesystem required

Two approaches to forensic extraction

Filesystem-Aware (TSK)

Superblock | Inode Table | Data Blocks

↓

Files & Directories

✓ File context, paths, timestamps

✗ Misses deleted/unallocated/compressed

Stream-Based (bulk_extractor)

Raw byte stream → → → → → →

↓

Emails, URLs, CCNs, ...

✓ Scans everything: slack, unalloc, compressed, swap

✗ No file paths, no directory context

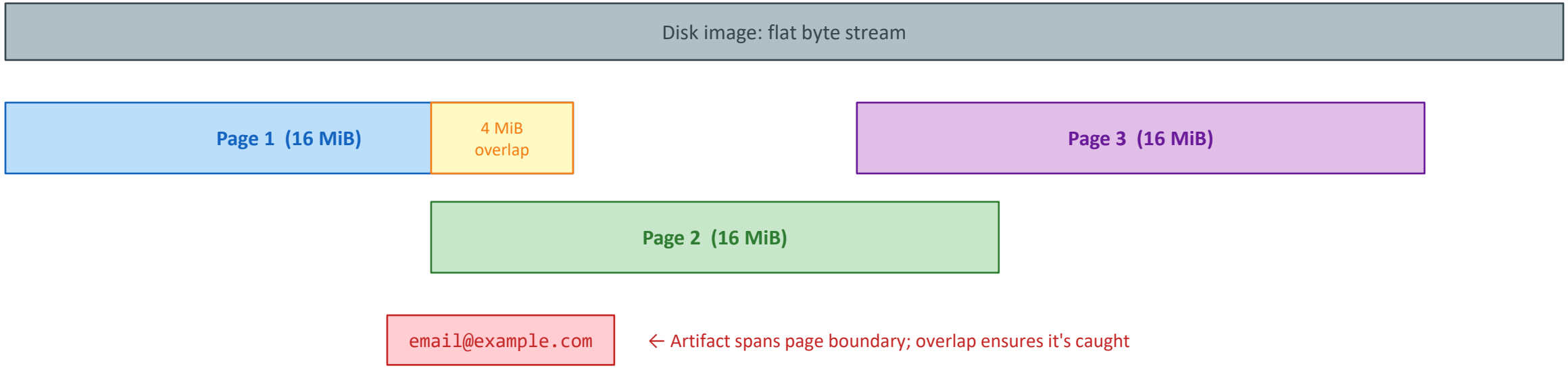
BE finds things TSK can't see; TSK gives context BE can't provide.

Benefits of stream-based processing

- Speed
  - parallel page processing across all CPU cores
  - typically 1+ GB/sec on modern hardware
- Thoroughness
  - scans slack space, unallocated blocks, swap partitions
  - secompresses and re-scans GZIP, ZIP, BASE64, hibernation files
- Generality
  - works on any media: raw images, disk dumps, memory captures, network pcaps
  - no filesystem driver required — treats input as a flat byte stream

The Page Model

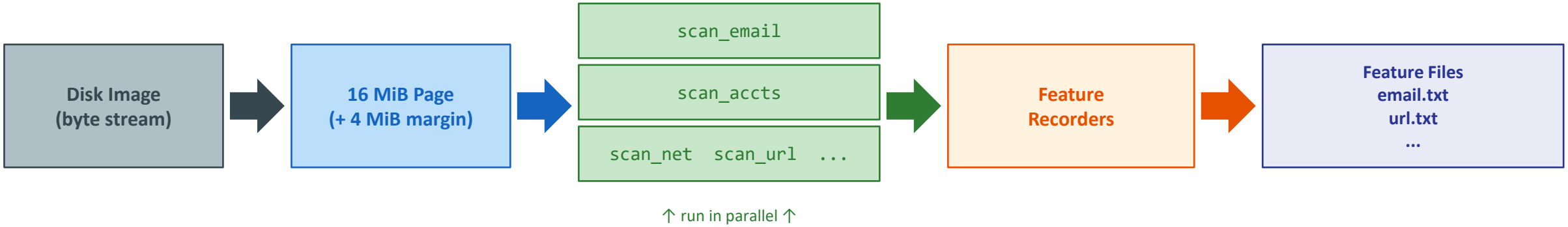
BE divides the input into overlapping pages for parallel processing.



Each page is processed independently by a thread. The 4 MiB margin ensures no artifact is split.

SCANNER PIPELINE

Each page is dispatched to all enabled scanners in parallel. Hits are written to feature files.



Recursive ➔ if a scanner detects compressed/encoded data, it decompresses and re-scans

Recursive decompression

Scanners detect embedded compressed/encoded data and re-scan it

- GZIP, ZIP, BASE64, HIBER (Windows hibernation) are auto-detected
- Decompressed content is fed back through the full scanner pipeline
- Enables discovery of artifacts nested inside archives

Forensic path notation records the decompression chain:

`1048576-GZIP-0`

» = byte 0 inside GZIP blob starting at image offset 1048576

`2097152-ZIP-4096-BASE64-128`

» = byte 128 inside BASE64 at offset 4096 inside ZIP at offset 2097152

The path is a **breadcrumb trail** back to the original bytes on disk

OUTPUT

feature files & histograms

Anatomy of a Feature File

Each feature file is a three-column, **tab**-separated text file:

```
forensic_path	feature	context
```

Example from email.txt:

```
49152	jdoe@example.com	From: John Doe <jdoe@example.com>
1048576-GZIP-0	admin@corp.net	Subject: credentials
```

- **forensic_path**
  - byte offset in the image (or decompression chain)
- **feature**
  - the extracted artifact (email address, CCN, URL, etc.)
- **context**
  - surrounding bytes for analyst interpretation (±16 bytes typical)

Forensic paths: reading the breadcrumb trail

Simple path:

`49152`

- » feature found at byte offset 49,152 in the raw image

Single decompression:

`1048576-GZIP-0`

- » GZIP blob found at offset 1,048,576; feature at byte 0 of decompressed stream

Nested decompression:

`1048576-GZIP-0-BASE64-128`

- » inside the GZIP stream, a BASE64 blob was found; feature at byte 128 of decoded data

Read **right-to-left** to trace the decompression chain from artifact back to disk offset

Core feature files

- PII / Financial
  - email.txt ccn.txt telephone.txt domain.txt
- Communication
  - url.txt rfc822.txt httplogs.txt
- Web / Network
  - ip.txt ether.txt url_searches.txt
- Media
  - exif.txt gps.txt jpeg_carved.txt
- OS Artifacts
  - winpe.txt winlnk.txt winprefetch.txt
- Utility
  - find.txt wordlist.txt json.txt

Histograms: frequency as signal

bulk_extractor auto-generates histogram files alongside feature files

Example: email_histogram.txt

```
n=1432   admin@company.com
n=845    noreply@service.com
n=12     suspect_alias@proton.me
n=3      burner47@tutanota.com
```

Interpretation:

- **High frequency** = likely noise (system accounts, cached headers, OS strings)
- **Low frequency** = potentially interesting (unique contacts, one-time communications)

Histograms are the **executive summary** of a scan — review these first

Carved Data & Special Files

Carved output directories:

- `jpeg/` — carved JPEG images (can view with image viewer, check EXIF)
- `unzip/` — contents extracted from ZIP archives found in the stream
- `sqlite/` — carved SQLite databases (browser history, app data)

Metadata & diagnostics:

- `report.xml` — DFXML provenance: command line, version, scan times, parameters
- `alerts.txt` — unusual findings (e.g., SSN patterns, encrypted data indicators)
- `stopped.txt` — scanners disabled due to errors or excessive output

Always check `report.xml` to verify scan completeness and parameters

SCANNERS

Scanner categories

- Pattern / Regex scanners
  - `scan_email`, `scan_accts`, `scan_telephone` — regular expression matching
- Structure-aware scanners
  - `scan_exif`, `scan_pdf`, `scan_sqlite` — parse binary formats
- Network scanners
  - `scan_net`, `scan_httplogs` — extract network artifacts from pcaps/memory
- Decompression / Recursive scanners
  - `scan_gzip`, `scan_zip`, `scan_base64`, `scan_hiberfile`
- Utility scanners
  - `scan_find` (custom patterns), `scan_wordlist` (build password dictionary)

Controlling scanners

- `-e scanner` — enable a disabled-by-default scanner
- `-x scanner` — disable a scanner
- `-E scanner` — enable only this scanner (disable all others)
- `-S param=value` — set a scanner-specific parameter
- `-H` — list all scanners and their parameters

Example: disable EXIF, enable only email + accts

```
bulk_extractor -x exif -o out image.raw
bulk_extractor -E email -E accts -o out image.raw
```

Use `-H` to discover tunable parameters for each scanner

Targeted Scanning Strategies

- Full triage (all scanners)
  - default mode — comprehensive but slower
  - appropriate for first-pass analysis or small images
- **Targeted scan (subset of scanners)**
  - use **-E** to enable only relevant scanners
  - dramatically faster for focused investigations
  - example: financial fraud case → enable only scan_accts
- **Custom pattern search**
  - `-f pattern` — search for a single literal string or regex
  - `-F file` — search for all patterns listed in a file (one per line)
  - results appear in find.txt feature file

The find Scanner & Custom Patterns

Use case: search for suspect-specific terms across entire media

Inline pattern:

```
bulk_extractor -f "darkweb_alias" -o out image.raw
```

Pattern file:

```
$ cat patterns.txt
john_alias_42
burner47@tutanota.com
192\.168\.1\.100
bulk_extractor -F patterns.txt -o out image.raw
```

Output: find.txt

- Same three-column format: forensic_path / matched pattern / context
- Works across compressed and encoded data (recursive scanning applies)

RUNNING bulk_extractor

Basic Invocation & Key Flags

Basic command:

```
bulk_extractor -o output_dir image.raw
```

Key flags:

- `-o dir` — output directory (must not exist)
- `-j N` — number of threads (default: CPU count)
- `-Y offset[-end]` — scan only a byte range (partial scan)
- `-R dir` — recursively scan all files in a directory
- `-w stoplist` — suppress known-good features

Three processing phases:

1. Scan pages in parallel (the bulk of processing time)
2. Generate histograms from feature files
3. Write report.xml with provenance metadata

Partial Scans & Performance

Partial scans with -Y

```
bulk_extractor -Y 0-10G -o triage_out image.raw
```

- » scan only the first 10 GB for rapid triage
- » useful for initial assessment before committing to full scan

Thread scaling:

`-j 1` — baseline (single-threaded)

`-j 4` — ~3.5x speedup (typical)

`-j 8` — ~6x speedup

`-j 16` — diminishing returns (I/O bound)

When to use partial scans:

- ◊ time-critical triage of large media (500 GB+)
- ◊ testing scanner configurations before committing to full run
- ◊ focusing on specific byte ranges identified by other tools

POST-PROCESSING

Stop lists: Filtering the noise

A stop list removes known-good features from results

Create a stop list from a clean reference image:

```
bulk_extractor -o clean_out clean_install.raw
```

Re-scan the suspect image with the stop list:

```
bulk_extractor -w clean_out -o suspect_out suspect.raw
```

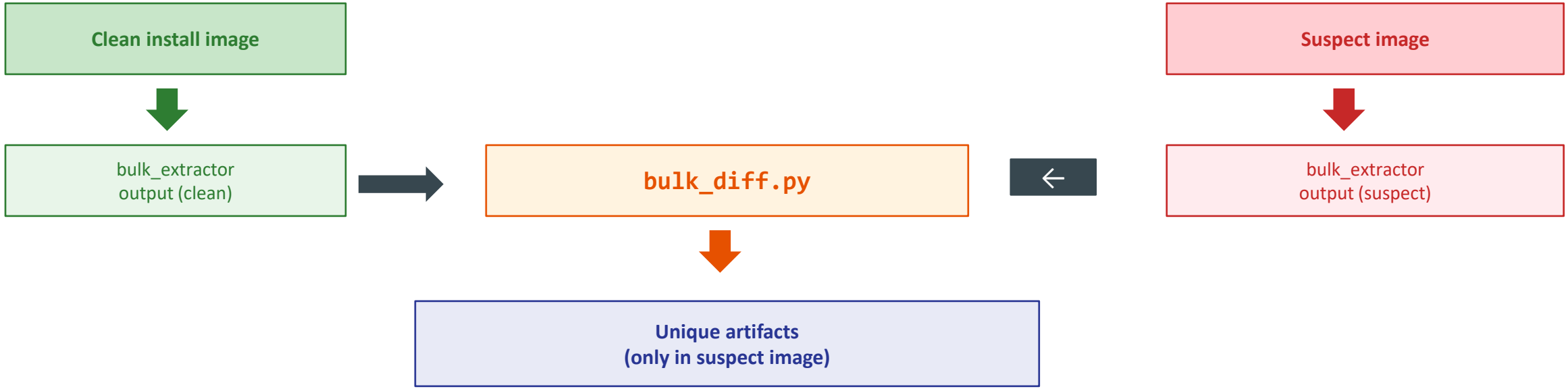
Result:

- Features present in the clean install are suppressed
- Only user-generated or suspect-specific artifacts remain

Same principle as block hash filtering:
known → **ignore**, focus on the unknown

Differential Analysis with bulk_diff.py

Compare two scans to isolate what changed or what's unique to the suspect image.



Differential analysis isolates user-generated content from OS baseline.

IDENTIFY_FILENAMES.PY: BRIDGING TO CONTEXT

Bridges the gap between stream-based scanning and filesystem context

Workflow:

1. Run `fiwalk` on the image to produce DFXML (file-to-offset mapping)
2. Run `identify_filenames.py` to annotate feature files with filenames

Before:

```
49152    jdoe@example.com    From: John Doe ...
```

After:

```
49152    jdoe@example.com    From: John Doe ...    /home/user/sent.mbox
```

Critical integration point:

- Answers:
  - Which file contained this email?
  - Was it in a deleted file?
- Offsets beyond allocated space → feature was in unallocated/deleted data

Combines BE's thoroughness with TSK's contextual power

Working with Feature Files: grep, sort, awk

Count unique email addresses:

```
cut -f2 email.txt | sort -u | wc -l
```

Extract .ru domain emails:

```
grep '\.ru' email.txt | cut -f2 | sort -u
```

Find features from unallocated space (offset > partition end):

```
awk -F'\t' '$1 > 104857600' email.txt
```

Cross-reference: which files contain credit card numbers?

```
cut -f4 annotated_ccn.txt | sort | uniq -c | sort -rn
```

Top 10 most-contacted email domains:

```
cut -f2 email.txt | grep -oP '@\K[^\t]+' | sort | uniq -c | sort -rn | head
```

PUTTING IT ALL TOGETHER

BULK_EXTRACTOR IN THE FORENSIC WORKFLOW

Full investigative pipeline from acquisition through reporting:

BE runs early for fast triage. identify_filenames.py connects BE hits to TSK context.

Plaso incorporates BE output into the unified timeline for temporal analysis.

No single tool is sufficient. The workflow chains complementary tools for comprehensive analysis.

WHAT BULK_EXTRACTOR CAN**NOT** DO

- No file attribution (alone)
  - features have byte offsets, not filenames — need identify_filenames.py + fiwalk
- No directory trees
  - stream-based scanning has no concept of directories or file hierarchy
- No timeline
  - no temporal ordering of artifacts — need Plaso or TSK mactime for that
- No user intent
  - finding an email address does not prove the user sent or read it
  - context, corroboration, and complementary tools are always required

BE is a **triage and discovery** tool, not a complete forensic solution

Summary & Tutorial Preview

Key takeaways:

- ◊ Stream-based scanning finds artifacts that filesystem-aware tools miss
- ◊ Feature files + histograms provide fast, structured triage output
- ◊ Forensic paths trace every artifact back to its location on disk
- ◊ Post-processing (stop lists, diff, identify_filenames) refines raw output
- ◊ Integration with TSK and Plaso gives the complete forensic picture

Companion tutorial:

- ◊ Hands-on lab with a prepared 128 MB ext3 image
- ◊ 13 progressive tasks: full scan → feature analysis → targeted scanning → post-processing
- ◊ Includes stop list creation, identify_filenames integration, and capstone triage report